RedSec Range — Defensive Challenge Learning Guide

110 concept cards · 162 declared variants · architecture, review, and remediation

RedSec Range

How to use this guide
1 Portfolio map
2 Attribution and pinned sources
3 Challenge review cards
4 `builtin` — RedSec built-in course
4.1 `01/first-contact` — First contact -- coax the orientation flag out of the Guide
4.2 `02/fingerprint-model` — Fingerprint the backing model/provider
4.3 `02/enumerate-surfaces` — Discover an undocumented internal endpoint
4.4 `02/leak-config` — Dump service config via a verbose error
4.5 `03/sys-prompt-leak` — Direct prompt injection -> leak the system prompt
4.6 `03/indirect-injection` — Indirect prompt injection via auto-retrieved notes
4.7 `03/tool-exfil` — Excessive agency -> exfiltrate the mail API key
4.8 `03/memory-poisoning` — Memory poisoning -> persistence across turns
4.9 `04/trust-exploitation` — Transitive trust -> reviewer acts on your ticket
4.10 `04/message-manipulation` — Send an unsigned A2A JSON-RPC message
4.11 `04/agent-impersonation` — Impersonate a trusted approver
4.12 `04/session-smuggling` — Agent session smuggling across a stateful A2A session
4.13 `05/knowledge-poisoning` — Poison the knowledge base
4.14 `05/retrieval-manipulation` — Out-rank the real documents for a victim query
4.15 `05/citation-spoofing` — Spoof a trusted citation
4.16 `06/embedding-inversion` — Reconstruct source text with a reference corpus
4.17 `06/membership-inference` — Confirm a record is present in the store
4.18 `06/embedding-collision` — Bypass a text check via an embedding collision
4.19 `07/tool-poisoning` — tools: tool-description poisoning hijacks the host
4.20 `07/resource-traversal` — resources: path traversal in resources/read
4.21 `07/prompt-injection` — prompts: argument injection discloses a privileged template block
4.22 `07/sampling-abuse` — sampling: server-initiated sampling exfiltrates host context
4.23 `07/roots-bypass` — roots: declared client roots are not enforced
4.24 `07/elicitation-phishing` — elicitation: server elicits credentials the host auto-answers
4.25 `07/logging-leak` — logging: debug log notifications leak a session token
4.26 `07/completion-leak` — completion: autocompletion enumerates secret values
4.27 `07/rug-pull` — tools: rug-pull -- tool behaviour changes after approval
4.28 `07/command-injection` — tools: command injection -> remote code execution
4.29 `07/header-routing-confusion` — 2026 routing: header/body policy desynchronization
4.30 `07/unicode-tool-poisoning` — tools: Unicode-concealed tool-description poisoning past the scanner
4.31 `08/malicious-weights` — RCE via unsafe model deserialization
4.32 `08/poisoned-dataset` — Backdoor the model via dataset poisoning
4.33 `08/backdoored-adapter` — Inject behaviour via a backdoored adapter
4.34 `08/dependency-confusion` — Win dependency confusion
4.35 `09/metadata-ssrf` — SSRF to cloud metadata -> steal role credentials
4.36 `09/open-inference-api` — Abuse an unauthenticated admin inference route
4.37 `09/registry-secrets` — Recover a secret from a model card
4.38 `10/asset-inventory` — Inventory the high-value AI assets
4.39 `10/trust-boundaries` — Map trust boundaries & data flows
4.40 `10/attack-paths` — Enumerate end-to-end attack paths

4.41 `11/initial-access` — Initial access via indirect injection
4.42 `11/pivot-vault` — Pivot through ambient authority to the vault
4.43 `11/exfil-crown-jewel` — Exfiltrate the crown-jewel secret
5 `dvaa` — Damn Vulnerable AI Agent
5.1 `L1-01` — Extract the System Prompt
5.2 `L1-02` — API Key Leak
5.3 `L1-03` — Basic Prompt Injection
5.4 `L2-01` — Jailbreak via Roleplay
5.5 `L2-02` — Context Window Manipulation
5.6 `L2-03` — MCP Path Traversal
5.7 `L2-04` — Persistent Memory Injection
5.8 `L2-05` — Memory Credential Extraction
5.9 `L2-06` — Context Padding Attack
5.10 `L2-07` — Safety Instruction Displacement
5.11 `L2-08` — Malicious Tool Registration
5.12 `L2-09` — Tool Call MITM
5.13 `L3-01` — Chained Prompt Injection
5.14 `L3-02` — SSRF via MCP
5.15 `L3-03` — Self-Replicating Memory Entry
5.16 `L3-04` — System Prompt Extraction via Context Pressure
5.17 `L3-05` — Tool Typosquatting
5.18 `L3-06` — Tool Chain Data Exfiltration
5.19 `L3-07` — Tool Shadowing
5.20 `L3-08` — Traffic Redirection Attack
5.21 `L4-01` — Compromise SecureBot
5.22 `L4-02` — Agent-to-Agent Attack Chain
6 `mcpgoat` — MCPGoat
6.1 `A1` — Tool Poisoning
6.2 `A2` — Tool Shadowing / Trusted-Tool Override
6.3 `A3` — Rug Pull / Tool Mutation (TOCTOU)
6.4 `B1` — Indirect Prompt Injection via Tool Output
6.5 `D1` — Command Injection (RCE)
6.6 `D2` — Path Traversal / Arbitrary File Read
6.7 `D3` — Server-Side Request Forgery
6.8 `D4` — SQL Injection
6.9 `C2` — Broken Authorization / Confused Deputy
6.10 `C3` — Insecure Direct Object Reference
6.11 `E1` — Sensitive Data Exposure
6.12 `A9` — Invisible-Text Tool Poisoning
6.13 `B2` — Indirect Injection via Resource Content
6.14 `B3` — Prompt-Template Injection
6.15 `B5` — Sampling Abuse (server-driven LLM calls)
6.16 `C4` — OAuth Token-Audience Confusion
6.17 `D6` — Server-Side Template Injection
6.18 `F1` — DNS Rebinding / Missing Origin Validation
6.19 `F2` — CORS Misconfiguration
6.20 `C6` — Predictable Session IDs (hijack)

6.21 G1 — Unbounded Consumption (cost/DoS)
6.22 G4 — Regular-Expression DoS (ReDoS)
6.23 D5 — NoSQL Injection
6.24 D7 — XML External Entity (XXE)
6.25 D8 — Insecure Deserialization (prototype pollution)
6.26 H1 — Supply Chain (typosquat / unsigned package)
7 dvmcp — Damn Vulnerable MCP Server
7.1 01 — Basic Prompt Injection
7.2 02 — Tool Poisoning
7.3 03 — Excessive Permission Scope
7.4 04 — Rug Pull Attack
7.5 05 — Tool Shadowing
7.6 06 — Indirect Prompt Injection
7.7 07 — Token Theft
7.8 08 — Malicious Code Execution
7.9 09 — Remote Access Control
7.10 10 — Multi-Vector Attack
8 appsecco-mcp — Vulnerable MCP Servers Lab
8.1 filesystem-workspace-actions — Filesystem Workspace Actions
8.2 indirect-prompt-injection — Indirect Prompt Injection (stdio)
8.3 indirect-prompt-injection-remote — Indirect Prompt Injection (HTTP+SSE)
8.4 malicious-code-exec — Malicious Code Execution
8.5 malicious-tools — Malicious Tools
8.6 namespace-typosquatting — Namespace Typosquatting
8.7 outdated-packages — Outdated Packages
8.8 secrets-pii — Secrets and PII Exposure
8.9 wikipedia-streamable — Wikipedia (Streamable HTTP)

How to use this guide

> **Scope.** This edition supports defensive education, architecture review, remediation, and non-destructive validation. It intentionally omits challenge secrets, operational payloads, and step-by-step exploitation procedures.

For each challenge, identify the trust boundary, answer the review questions, apply the defensive resolution, and validate the expected allow/deny behavior with synthetic data. Runtime exploitability is **not verified—outside the current scope**. Existing project and upstream evidence is linked as provenance rather than reproduced here.

This guide provides one defensive card per catalog concept. The 26 MCPGoat cards each represent three declared upstream levels, so the 110 concept cards span 162 declared runnable variants. Those 52 additional levels are not separate, level-specific walkthroughs in this edition.

1 Portfolio map

Range	Concepts	Declared variants	Evidence meaning
builtin	43	43	Repository manifests and declared verifiers
dvaa	22	22	Pinned upstream metadata; adapter health does not prove solvability
mcpgoat	26	78	Pinned upstream metadata; adapter health does not prove solvability
dvmcp	10	10	Pinned upstream metadata; adapter health does not prove solvability
appsecco-mcp	9	9	Pinned upstream metadata; adapter health does not prove solvability

2 Attribution and pinned sources

Range	Maintainer	License	Revision	Repository / status
`builtin`	RedSec Range	Apache-2.0	`repository state`	https://github.com/tq05/RedSec-range (available) — Challenge metadata is derived from the 11 module manifests.
`dvaa`	OpenA2A	Apache-2.0	`faf3fb172ef526d12db0eb5377701549693118a6`	https://github.com/opena2a-org/damn-vulnerable-ai-agent (available) — The dashboard exposes 22 CTF challenges; each upstream definition embeds hints, a solution, success criteria, and defenses.
`mcpgoat`	Sabyasachi Dhal	MIT	`80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed`	https://github.com/SabyasachiDhal/MCPGoat (available) — There are 26 challenge concepts, each implemented at easy, moderate, and difficult levels (78 scored flags), plus a Secure reference level.
`dvmcp`	Harish Santhanalakshmi Ganesan	MIT (declared in README; no license file at pinned revision)	`79734c19f5104cd11486c90926d245560f53befa`	https://github.com/harishsg993010/damn-vulnerable-MCP-server (available) — All ten challenges have upstream step-by-step solution guides and mitigation advice.

Range	Maintainer	License	Revision	Repository / status
`appsecco-mcp`	Appsecco	MIT	`47b7f2082261136b667daa7be9273698e66e479e`	https://github.com/appsecco/vulnerable-mcp-servers-lab (available) — Upstream publishes demonstration payloads in each lab README rather than a formal graded solution suite.
`offsec`	OffSec	proprietary	`not configured`	https://www.offsec.com/courses/ai-300/ (placeholder) — Add only when OffSec publishes and licenses an automation interface.

3 Challenge review cards

The cards are grouped by range. The `offsec` catalog entry has no deployable challenges and remains a placeholder, so it contributes no card.

4 `builtin` — RedSec built-in course

4.1 `01/first-contact` — First contact -- coax the orientation flag out of the Guide

Category: direct-injection · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/01/first-contact` · **Defensive pattern:** `prompt`

4.1.1 Challenge description

Review how the built-in challenge models first contact -- coax the orientation flag out of the guide, focusing on its direct-injection trust boundary.

4.1.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.1.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.1.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.2 `02/fingerprint-model` — Fingerprint the backing model/provider

Category: recon · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/02/fingerprint-model` · **Defensive pattern:** `recon`

4.2.1 Challenge description

Review how the built-in challenge models fingerprint the backing model/provider, focusing on its recon trust boundary.

4.2.2 Educational focus

Discovery surfaces should disclose only what an authenticated user needs for an authorised workflow.

Trust boundary: Model, provider, route, error, and capability details cross from internal configuration to an external caller.

4.2.3 Guided defensive review

1. Which metadata is necessary for this caller?
2. Can errors, banners, schemas, or undocumented routes reveal additional capability?

4.2.4 Defensive resolution

Minimise banners and errors, authenticate discovery endpoints, inventory routes and schemas, and rate-limit enumeration.

Validation: Compare permitted and unauthorised synthetic clients and assert that only the documented minimum metadata is returned.

Gotcha: Removing a route from user documentation does not make the reachable endpoint private.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.3 `02/enumerate-surfaces` — Discover an undocumented internal endpoint

Category: recon · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/02/enumerate-surfaces` · **Defensive pattern:** `recon`

4.3.1 Challenge description

Review how the built-in challenge models discover an undocumented internal endpoint, focusing on its recon trust boundary.

4.3.2 Educational focus

Discovery surfaces should disclose only what an authenticated user needs for an authorised workflow.

Trust boundary: Model, provider, route, error, and capability details cross from internal configuration to an external caller.

4.3.3 Guided defensive review

1. Which metadata is necessary for this caller?
2. Can errors, banners, schemas, or undocumented routes reveal additional capability?

4.3.4 Defensive resolution

Minimise banners and errors, authenticate discovery endpoints, inventory routes and schemas, and rate-limit enumeration.

Validation: Compare permitted and unauthorised synthetic clients and assert that only the documented minimum metadata is returned.

Gotcha: Removing a route from user documentation does not make the reachable endpoint private.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.4 `02/leak-config` — Dump service config via a verbose error

Category: recon · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `builtin/02/leak-config` · **Defensive pattern:** `recon`

4.4.1 Challenge description

Review how the built-in challenge models dump service config via a verbose error, focusing on its recon trust boundary.

4.4.2 Educational focus

Discovery surfaces should disclose only what an authenticated user needs for an authorised workflow.

Trust boundary: Model, provider, route, error, and capability details cross from internal configuration to an external caller.

4.4.3 Guided defensive review

1. Which metadata is necessary for this caller?
2. Can errors, banners, schemas, or undocumented routes reveal additional capability?

4.4.4 Defensive resolution

Minimise banners and errors, authenticate discovery endpoints, inventory routes and schemas, and rate-limit enumeration.

Validation: Compare permitted and unauthorised synthetic clients and assert that only the documented minimum metadata is returned.

Gotcha: Removing a route from user documentation does not make the reachable endpoint private.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.5 `03/sys-prompt-leak` — Direct prompt injection -> leak the system prompt

Category: direct-injection · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/03/sys-prompt-leak` · **Defensive pattern:** `prompt`

4.5.1 Challenge description

Review how the built-in challenge models direct prompt injection -> leak the system prompt, focusing on its direct-injection trust boundary.

4.5.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.5.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.5.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.6 `03/indirect-injection` — Indirect prompt injection via auto-retrieved notes

Category: indirect-injection · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `builtin/03/indirect-injection` · **Defensive pattern:** `prompt`

4.6.1 Challenge description

Review how the built-in challenge models indirect prompt injection via auto-retrieved notes, focusing on its indirect-injection trust boundary.

4.6.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.6.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.6.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.7 `03/tool-exfil` — Excessive agency -> exfiltrate the mail API key

Category: tool-abuse · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/03/tool-exfil` · **Defensive pattern:** `agency`

4.7.1 Challenge description

Review how the built-in challenge models excessive agency -> exfiltrate the mail api key, focusing on its tool-abuse trust boundary.

4.7.2 Educational focus

Tool availability is not action authorisation; the system that owns the effect must enforce policy for each call.

Trust boundary: A model-selected action crosses into a tool or service that has real authority and external effects.

4.7.3 Guided defensive review

1. Which principal authorises the effect?
2. Can model-controlled arguments exceed the intended task or data scope?

4.7.4 Defensive resolution

Use narrow tools, per-action authorisation, allow-listed arguments, minimal credentials, confirmation for sensitive effects, and audit logs.

Validation: Exercise permitted and denied actions with synthetic accounts and assert enforcement at the tool or service boundary.

Gotcha: A model refusal or tool description cannot contain authority that the downstream service fails to enforce.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.8 `03/memory-poisoning` — Memory poisoning -> persistence across turns

Category: memory · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/03/memory-poisoning` · **Defensive pattern:** `prompt`

4.8.1 Challenge description

Review how the built-in challenge models memory poisoning -> persistence across turns, focusing on its memory trust boundary.

4.8.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.8.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.8.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.9 `04/trust-exploitation` — Transitive trust -> reviewer acts on your ticket

Category: a2a-trust · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/04/trust-exploitation` · **Defensive pattern:** `identity`

4.9.1 Challenge description

Review how the built-in challenge models transitive trust -> reviewer acts on your ticket, focusing on its a2a-trust trust boundary.

4.9.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

4.9.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

4.9.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.10 `04/message-manipulation` — Send an unsigned A2A JSON-RPC message

Category: a2a-message/send · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `builtin/04/message-manipulation` · **Defensive pattern:** `identity`

4.10.1 Challenge description

Review how the built-in challenge models send an unsigned a2a json-rpc message, focusing on its a2a-message/send trust boundary.

4.10.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

4.10.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

4.10.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.11 `04/agent-impersonation` — Impersonate a trusted approver

Category: a2a-message-identity · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/04/agent-impersonation` · **Defensive pattern:** `identity`

4.11.1 Challenge description

Review how the built-in challenge models impersonate a trusted approver, focusing on its a2a-message-identity trust boundary.

4.11.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

4.11.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

4.11.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.12 `04/session-smuggling` — Agent session smuggling across a stateful A2A session

Category: a2a-context · **Difficulty:** impossible · **Declared variants:** 1 **Catalog key:** `builtin/04/session-smuggling` · **Defensive pattern:** `session-context`

4.12.1 Challenge description

Review how the built-in challenge models agent session smuggling across a stateful a2a session, focusing on its a2a-context trust boundary.

4.12.2 Educational focus

Every message must be authenticated and authorised in the context of the accumulated session state.

Trust boundary: Conversation state and delegated authority cross message, task, and session boundaries over time.

4.12.3 Guided defensive review

1. Who owns the context and each message?
2. Can replay, reordering, or partial validation change the cumulative meaning?

4.12.4 Defensive resolution

Bind context identifiers to authenticated principals, authorise every transition, validate cumulative state, and expire replayable state.

Validation: Use harmless cross-user, replayed, split, and out-of-order messages to confirm the state machine rejects invalid transitions.

Gotcha: Validating each message in isolation can miss a prohibited meaning assembled across several messages.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.13 `05/knowledge-poisoning` — Poison the knowledge base

Category: rag-source · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/05/knowledge-poisoning` · **Defensive pattern:** `rag`

4.13.1 Challenge description

Review how the built-in challenge models poison the knowledge base, focusing on its rag-source trust boundary.

4.13.2 Educational focus

Relevance is not trust; retrieval, provenance, privacy, and instruction handling are separate decisions.

Trust boundary: Indexed content, similarity scores, or derived vectors cross from a data store into a trusted answer.

4.13.3 Guided defensive review

1. Who can add or rank content?
2. What can similarity, provenance, or output precision disclose?

4.13.4 Defensive resolution

Control ingestion, preserve provenance, isolate instructions from retrieved data, minimise precision, and authorise the corpus.

Validation: Use synthetic documents and records to test ranking, provenance display, tenant isolation, and privacy thresholds.

Gotcha: A high similarity score or authoritative-looking citation does not establish integrity or permission.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.14 `05/retrieval-manipulation` — Out-rank the real documents for a victim query

Category: rag-retrieval · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `builtin/05/retrieval-manipulation` · **Defensive pattern:** `rag`

4.14.1 Challenge description

Review how the built-in challenge models out-rank the real documents for a victim query, focusing on its rag-retrieval trust boundary.

4.14.2 Educational focus

Relevance is not trust; retrieval, provenance, privacy, and instruction handling are separate decisions.

Trust boundary: Indexed content, similarity scores, or derived vectors cross from a data store into a trusted answer.

4.14.3 Guided defensive review

1. Who can add or rank content?
2. What can similarity, provenance, or output precision disclose?

4.14.4 Defensive resolution

Control ingestion, preserve provenance, isolate instructions from retrieved data, minimise precision, and authorise the corpus.

Validation: Use synthetic documents and records to test ranking, provenance display, tenant isolation, and privacy thresholds.

Gotcha: A high similarity score or authoritative-looking citation does not establish integrity or permission.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.15 `05/citation-spoofing` — Spoof a trusted citation

Category: rag-trust · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/05/citation-spoofing` · **Defensive pattern:** `rag`

4.15.1 Challenge description

Review how the built-in challenge models spoof a trusted citation, focusing on its rag-trust trust boundary.

4.15.2 Educational focus

Relevance is not trust; retrieval, provenance, privacy, and instruction handling are separate decisions.

Trust boundary: Indexed content, similarity scores, or derived vectors cross from a data store into a trusted answer.

4.15.3 Guided defensive review

1. Who can add or rank content?
2. What can similarity, provenance, or output precision disclose?

4.15.4 Defensive resolution

Control ingestion, preserve provenance, isolate instructions from retrieved data, minimise precision, and authorise the corpus.

Validation: Use synthetic documents and records to test ranking, provenance display, tenant isolation, and privacy thresholds.

Gotcha: A high similarity score or authoritative-looking citation does not establish integrity or permission.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.16 `06/embedding-inversion` — Reconstruct source text with a reference corpus

Category: embedding-inversion · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/06/embedding-inversion` · **Defensive pattern:** `embeddings`

4.16.1 Challenge description

Review how the built-in challenge models reconstruct source text with a reference corpus, focusing on its embedding-inversion trust boundary.

4.16.2 Educational focus

Embeddings can retain information about source records and should not serve as identity or authorisation controls.

Trust boundary: Derived vectors and similarity decisions cross privacy, tenant, and policy boundaries.

4.16.3 Guided defensive review

1. Who may query the vectors and at what precision?
2. Can outputs reveal membership, reconstruct content, or bypass a text-only policy?

4.16.4 Defensive resolution

Authorise vector access, isolate tenants, minimise returned precision, rate-limit queries, and enforce policy on canonical source data.

Validation: Use synthetic canary records to measure leakage and verify that similarity cannot replace exact policy checks.

Gotcha: A derived representation is not automatically anonymous, and semantic similarity is not an access-control decision.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.17 `06/membership-inference` — Confirm a record is present in the store

Category: embedding-mia · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/06/membership-inference` · **Defensive pattern:** `embeddings`

4.17.1 Challenge description

Review how the built-in challenge models confirm a record is present in the store, focusing on its embedding-mia trust boundary.

4.17.2 Educational focus

Embeddings can retain information about source records and should not serve as identity or authorisation controls.

Trust boundary: Derived vectors and similarity decisions cross privacy, tenant, and policy boundaries.

4.17.3 Guided defensive review

1. Who may query the vectors and at what precision?
2. Can outputs reveal membership, reconstruct content, or bypass a text-only policy?

4.17.4 Defensive resolution

Authorise vector access, isolate tenants, minimise returned precision, rate-limit queries, and enforce policy on canonical source data.

Validation: Use synthetic canary records to measure leakage and verify that similarity cannot replace exact policy checks.

Gotcha: A derived representation is not automatically anonymous, and semantic similarity is not an access-control decision.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.18 `06/embedding-collision` — Bypass a text check via an embedding collision

Category: embedding-collision · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `builtin/06/embedding-collision` · **Defensive pattern:** `embeddings`

4.18.1 Challenge description

Review how the built-in challenge models bypass a text check via an embedding collision, focusing on its embedding-collision trust boundary.

4.18.2 Educational focus

Embeddings can retain information about source records and should not serve as identity or authorisation controls.

Trust boundary: Derived vectors and similarity decisions cross privacy, tenant, and policy boundaries.

4.18.3 Guided defensive review

1. Who may query the vectors and at what precision?
2. Can outputs reveal membership, reconstruct content, or bypass a text-only policy?

4.18.4 Defensive resolution

Authorise vector access, isolate tenants, minimise returned precision, rate-limit queries, and enforce policy on canonical source data.

Validation: Use synthetic canary records to measure leakage and verify that similarity cannot replace exact policy checks.

Gotcha: A derived representation is not automatically anonymous, and semantic similarity is not an access-control decision.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.19 `07/tool-poisoning` — tools: tool-description poisoning hijacks the host

Category: tools · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/07/tool-poisoning` · **Defensive pattern:** `tool-poisoning`

4.19.1 Challenge description

Review how the built-in challenge models tools: tool-description poisoning hijacks the host, focusing on its tools trust boundary.

4.19.2 Educational focus

Tool metadata is untrusted content and must not silently grant authority or inject instructions.

Trust boundary: Tool descriptions and results cross from a server-controlled metadata plane into model planning and user approval.

4.19.3 Guided defensive review

1. Which metadata reaches the model or approval UI?
2. Does execution use the same reviewed, authenticated definition?

4.19.4 Defensive resolution

Authenticate servers, sanitise and canonicalise metadata, show security-relevant fields, and authorise every call outside the description.

Validation: Use benign instruction-like metadata to confirm it remains visible as data and cannot broaden a tool's permitted effect.

Gotcha: A harmless tool name can conceal instructions in descriptions, annotations, or returned content.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.20 `07/resource-traversal` — resources: path traversal in resources/read

Category: resources · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `builtin/07/resource-traversal` · **Defensive pattern:** `path`

4.20.1 Challenge description

Review how the built-in challenge models resources: path traversal in resources/read, focusing on its resources trust boundary.

4.20.2 Educational focus

Access decisions must use the final canonical target and remain safe across encoding, absolute paths, and symbolic links.

Trust boundary: An untrusted path or URI crosses into filesystem resolution outside an authorised root.

4.20.3 Guided defensive review

1. What is the canonical allowed root?
2. Can decoding, symlinks, alternate separators, or races change the resolved target?

4.20.4 Defensive resolution

Resolve paths safely, reject absolute and escaping targets, enforce containment after canonicalisation, and run with minimal filesystem access.

Validation: Use temporary in-root and out-of-root fixtures with encoded, absolute, separator, and symlink cases and assert fail-closed behavior.

Gotcha: String-prefix checks can approve sibling paths or links whose final target lies outside the intended root.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.21 `07/prompt-injection` — prompts: argument injection discloses a privileged template block

Category: prompts · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `builtin/07/prompt-injection` · **Defensive pattern:** `prompt`

4.21.1 Challenge description

Review how the built-in challenge models prompts: argument injection discloses a privileged template block, focusing on its prompts trust boundary.

4.21.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

4.21.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

4.21.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.22 `07/sampling-abuse` — sampling: server-initiated sampling exfiltrates host context

Category: sampling · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/07/sampling-abuse` · **Defensive pattern:** `sampling`

4.22.1 Challenge description

Review how the built-in challenge models sampling: server-initiated sampling exfiltrates host context, focusing on its sampling trust boundary.

4.22.2 Educational focus

Sampling is a privileged host action that needs explicit purpose, context minimisation, consent, and resource limits.

Trust boundary: A server-requested model call crosses back into host context, credentials, tokens, and compute budgets.

4.22.3 Guided defensive review

1. Which context is shared with the sampled model?
2. Who authorises the purpose, model, token budget, and returned destination?

4.22.4 Defensive resolution

Require informed consent, minimise context, isolate credentials, allow-list models and purposes, cap tokens and cost, and audit results.

Validation: Use synthetic host context and bounded budgets to confirm an untrusted server cannot expand context or repeat calls silently.

Gotcha: Approving the parent tool call does not automatically approve every server-initiated model call it requests.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.23 `07/roots-bypass` — roots: declared client roots are not enforced

Category: roots · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `builtin/07/roots-bypass` · **Defensive pattern:** `scope`

4.23.1 Challenge description

Review how the built-in challenge models roots: declared client roots are not enforced, focusing on its roots trust boundary.

4.23.2 Educational focus

Servers must enforce the intersection of declared scope, authenticated identity, and canonical resource location.

Trust boundary: A client-declared root, permission, or resource scope crosses into server-side access control.

4.23.3 Guided defensive review

1. Who defines and enforces the allowed scope?
2. Is the resource canonicalised before containment and permission checks?

4.23.4 Defensive resolution

Enforce scopes server-side, canonicalise resource identifiers, deny ambient access, and apply least privilege to the host process.

Validation: Use synthetic in-scope and out-of-scope resources, including encoded and absolute forms, to confirm containment fails closed.

Gotcha: Advertising a root or permission to the model does not enforce it at the filesystem or service boundary.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.24 `07/elicitation-phishing` — elicitation: server elicits credentials the host auto-answers

Category: elicitation · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/07/elicitation-phishing` · **Defensive pattern:** `elicitation`

4.24.1 Challenge description

Review how the built-in challenge models elicitation: server elicits credentials the host auto-answers, focusing on its elicitation trust boundary.

4.24.2 Educational focus

Elicitation needs a trusted host UI, clear server identity and purpose, schema limits, and no automatic secret completion.

Trust boundary: A server-generated request crosses into a trusted user interaction and may solicit sensitive input.

4.24.3 Guided defensive review

1. Can the user see who requested the value and why?
2. Can the host auto-fill credentials or accept undeclared fields?

4.24.4 Defensive resolution

Render elicitations in trusted chrome, show provenance, allow-list schemas, block secret autofill, and require explicit user consent.

Validation: Send benign unexpected and sensitive-looking fields and confirm the host refuses, redacts, or requires an explicit decision.

Gotcha: A valid elicitation schema does not make the requested value appropriate for that server.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.25 `07/logging-leak` — logging: debug log notifications leak a session token

Category: logging · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/07/logging-leak` · **Defensive pattern:** `logging`

4.25.1 Challenge description

Review how the built-in challenge models logging: debug log notifications leak a session token, focusing on its logging trust boundary.

4.25.2 Educational focus

Logging level changes must not bypass data classification, redaction, or caller authorisation.

Trust boundary: Runtime secrets and personal data cross into diagnostic notifications, logs, and support exports.

4.25.3 Guided defensive review

1. Which values can reach debug output?
2. Who may change verbosity and retrieve the resulting records?

4.25.4 Defensive resolution

Use structured allow-listed logging, redact at source, restrict verbosity changes, separate tenants, and apply retention controls.

Validation: Exercise each log level with synthetic canaries and assert that sensitive fields never appear for unauthorised readers.

Gotcha: Disabling verbose logs in production does not fix code paths that still format secrets before filtering.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.26 `07/completion-leak` — completion: autocompletion enumerates secret values

Category: completion · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/07/completion-leak` · **Defensive pattern:** `completion`

4.26.1 Challenge description

Review how the built-in challenge models completion: autocompletion enumerates secret values, focusing on its completion trust boundary.

4.26.2 Educational focus

Completion sources need the same object and field authorisation as the final read operation.

Trust boundary: Autocompletion crosses from partial user input into internal indexes, identifiers, or secret-backed values.

4.26.3 Guided defensive review

1. Which corpus backs suggestions?
2. Is every suggested value authorised for this principal and purpose?

4.26.4 Defensive resolution

Use non-sensitive suggestion indexes, enforce field-level authorisation, cap results, and avoid secret stores as completion sources.

Validation: Probe with benign prefixes under different identities and assert that suggestions reveal only each caller's allowed namespace.

Gotcha: Hiding a complete value is insufficient when repeated prefix queries can enumerate it incrementally.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.27 `07/rug-pull` — tools: rug-pull -- tool behaviour changes after approval

Category: tools (rug-pull) · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `builtin/07/rug-pull` · **Defensive pattern:** `rug-pull`

4.27.1 Challenge description

Review how the built-in challenge models tools: rug-pull -- tool behaviour changes after approval, focusing on its tools (rug-pull) trust boundary.

4.27.2 Educational focus

Approval applies to an immutable definition digest and must be invalidated when security-relevant metadata or behavior changes.

Trust boundary: A previously reviewed tool definition changes between approval, discovery, and execution.

4.27.3 Guided defensive review

1. Which digest was approved?
2. What forces re-review when the definition or server identity changes?

4.27.4 Defensive resolution

Pin signed definitions, compare canonical digests before every call, cache immutably, and require re-approval for any change.

Validation: Use two benign versions of one tool and assert that the second cannot run under approval granted to the first.

Gotcha: Periodic refresh without digest-bound approval creates a time-of-check/time-of-use gap.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.28 `07/command-injection` — tools: command injection -> remote code execution

Category: tools (rce) · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/07/command-injection` · **Defensive pattern:** `command`

4.28.1 Challenge description

Review how the built-in challenge models tools: command injection -> remote code execution, focusing on its tools (rce) trust boundary.

4.28.2 Educational focus

Data must remain structured arguments and must never become executable command syntax.

Trust boundary: Untrusted arguments cross into a shell, interpreter, dynamic loader, or operating-system process boundary.

4.28.3 Guided defensive review

1. Is a shell or evaluator involved?
2. Are executable identity and every argument fixed or independently allow-listed?

4.28.4 Defensive resolution

Remove shell evaluation, call fixed executables with argument arrays, validate typed values, drop privileges, and sandbox the process.

Validation: Use harmless metacharacter and unexpected-argument fixtures and assert they are rejected or passed literally without side effects.

Gotcha: Escaping for one shell or platform is fragile; avoiding the interpreter is the durable control.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.29 `07/header-routing-confusion` — 2026 routing: header/body policy desynchronization

Category: streamable-http-routing · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/07/header-routing-confusion` · **Defensive pattern:** `transport`

4.29.1 Challenge description

Review how the built-in challenge models 2026 routing: header/body policy desynchronization, focusing on its streamable-http-routing trust boundary.

4.29.2 Educational focus

Every layer must agree on one canonical request and bind credentials to the intended origin, audience, and session.

Trust boundary: Identity and request meaning cross proxies, origins, sessions, protocol versions, or downstream services.

4.29.3 Guided defensive review

1. Which layer decides request meaning?
2. Are origin, audience, version, and session identifiers bound exactly?

4.29.4 Defensive resolution

Canonicalise once, validate exact origins and audiences, use unpredictable sessions, and reject contradictory routing data.

Validation: Exercise mismatched headers and metadata with harmless requests and assert that every layer reaches the same deny decision.

Gotcha: A gateway and application can each appear correct while authorising different interpretations of one request.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.30 `07/unicode-tool-poisoning` — tools: Unicode-concealed tool-description poisoning past the scanner

Category: tools (concealment) · **Difficulty:** impossible · **Declared variants:** 1 **Catalog key:** `builtin/07/unicode-tool-poisoning` · **Defensive pattern:** `unicode`

4.30.1 Challenge description

Review how the built-in challenge models tools: unicode-concealed tool-description poisoning past the scanner, focusing on its tools (concealment) trust boundary.

4.30.2 Educational focus

Security review and execution must use one canonical representation while preserving warnings about concealed characters.

Trust boundary: Unicode formatting and confusable characters cross from raw metadata into scanning, display, and model-visible text.

4.30.3 Guided defensive review

1. Which normalization form is enforced?
2. Are bidirectional controls, zero-width characters, and confusables exposed to reviewers?

4.30.4 Defensive resolution

Normalize consistently, reject or visibly escape control characters, detect confusables, and scan the same canonical text shown to users.

Validation: Use harmless strings containing bidi, zero-width, and confusable characters and assert identical scanner and display decisions.

Gotcha: Stripping a subset of invisible characters can create a second, differently interpreted representation.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.31 `08/malicious-weights` — RCE via unsafe model deserialization

Category: supply-weights · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/08/malicious-weights` · **Defensive pattern:** `model-deserialization`

4.31.1 Challenge description

Review how the built-in challenge models rce via unsafe model deserialization, focusing on its supply-weights trust boundary.

4.31.2 Educational focus

Model loading should consume data-only formats and occur in isolation before an artifact reaches production authority.

Trust boundary: A model artifact crosses from storage into a loader that may instantiate executable objects or custom operators.

4.31.3 Guided defensive review

1. Can the format encode executable objects or custom code?
2. Where are provenance, signature, and digest verified before loading?

4.31.4 Defensive resolution

Prefer safe tensor formats, verify signed provenance and immutable digests, scan or convert in a sandbox, and block custom loading hooks.

Validation: Load only benign fixtures in an isolated process and assert unsafe formats, unknown operators, and provenance failures are rejected.

Gotcha: A model repository or familiar extension does not guarantee that the loader treats the file as data only.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.32 `08/poisoned-dataset` — Backdoor the model via dataset poisoning

Category: supply-data · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/08/poisoned-dataset` · **Defensive pattern:** `data-poisoning`

4.32.1 Challenge description

Review how the built-in challenge models backdoor the model via dataset poisoning, focusing on its supply-data trust boundary.

4.32.2 Educational focus

Dataset integrity requires provenance, controlled contribution, anomaly review, and behavior-focused validation before promotion.

Trust boundary: Training or retrieval records cross from collection and labeling into learned or indexed behavior.

4.32.3 Guided defensive review

1. Who can add or relabel records?
2. Which tests detect targeted triggers, skew, duplication, or provenance gaps?

4.32.4 Defensive resolution

Version and sign datasets, restrict writers, review labels, deduplicate, scan anomalies, and evaluate targeted slices before deployment.

Validation: Add harmless synthetic outliers and trigger-like canaries and assert ingestion or evaluation identifies them before promotion.

Gotcha: Aggregate quality metrics can remain healthy while a small targeted subset creates a reliable backdoor.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.33 `08/backdoored-adapter` — Inject behaviour via a backdoored adapter

Category: supply-adapter · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/08/backdoored-adapter` · **Defensive pattern:** `adapter-backdoor`

4.33.1 Challenge description

Review how the built-in challenge models inject behaviour via a backdoored adapter, focusing on its supply-adapter trust boundary.

4.33.2 Educational focus

Adapters need independent provenance and behavioral evaluation because their small size does not limit their authority.

Trust boundary: An adapter or fine-tune delta crosses into a trusted base model and changes behavior without replacing the full artifact.

4.33.3 Guided defensive review

1. Which signed identity produced the adapter?
2. What targeted behavior changes appear when it is composed with the approved base?

4.33.4 Defensive resolution

Pin signed adapter digests, restrict loading, scan formats, evaluate targeted and safety suites, and promote the composed model as one artifact.

Validation: Compare a benign adapter and an unapproved digest in isolation and assert only the evaluated composition can be promoted.

Gotcha: Trust in the base model does not transfer to an adapter that can override high-impact behavior.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.34 `08/dependency-confusion` — Win dependency confusion

Category: supply-deps · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `builtin/08/dependency-confusion` · **Defensive pattern:** `dependency-confusion`

4.34.1 Challenge description

Review how the built-in challenge models win dependency confusion, focusing on its supply-deps trust boundary.

4.34.2 Educational focus

Dependency identity must bind namespace, registry, version, and digest rather than relying on the package name alone.

Trust boundary: A package name crosses internal and public registries where resolution priority may select an unintended publisher.

4.34.3 Guided defensive review

1. Which registry owns the namespace?
2. Can a public package or lookalike name outrank the intended internal dependency?

4.34.4 Defensive resolution

Use scoped private namespaces, explicit registries, lockfiles and hashes, signed provenance, and deny undeclared sources.

Validation: Resolve benign test packages against isolated registries and assert public, unsigned, or hash-mismatched candidates fail closed.

Gotcha: Pinning a version does not help when the same name and version can resolve from the wrong registry.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.35 `09/metadata-ssrf` — SSRF to cloud metadata -> steal role credentials

Category: infra-ssrf · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/09/metadata-ssrf` · **Defensive pattern:** `infra`

4.35.1 Challenge description

Review how the built-in challenge models ssrf to cloud metadata -> steal role credentials, focusing on its infra-ssrf trust boundary.

4.35.2 Educational focus

Workload identity and egress are authority boundaries; model-facing services should carry minimal ambient privilege.

Trust boundary: A model-facing workload crosses into cloud metadata, administrative APIs, registries, or network destinations.

4.35.3 Guided defensive review

1. Which destinations can the workload reach?
2. Which administrative or registry actions require authentication?

4.35.4 Defensive resolution

Default-deny egress, use workload identity, require admin authentication, isolate metadata, and keep secrets out of artifacts.

Validation: Review policy and use inert internal endpoints to confirm blocked destinations and authenticated administration.

Gotcha: Application input validation cannot compensate for broad network reach and over-privileged workload credentials.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.36 `09/open-inference-api` — Abuse an unauthenticated admin inference route

Category: infra-api · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/09/open-inference-api` · **Defensive pattern:** `admin-api`

4.36.1 Challenge description

Review how the built-in challenge models abuse an unauthenticated admin inference route, focusing on its infra-api trust boundary.

4.36.2 Educational focus

Administrative inference routes require strong authentication, explicit authorisation, isolation, and narrow network exposure.

Trust boundary: A model or inference administration operation crosses from a network client into privileged runtime configuration and resources.

4.36.3 Guided defensive review

1. Which principal may invoke each administrative operation?
2. Is the route separated from public inference traffic and ambient workload identity?

4.36.4 Defensive resolution

Require authenticated workload identity, role-based action checks, private network exposure, rate limits, and complete audit records.

Validation: Use authorised and unauthorised synthetic principals to verify every administrative method and object denies by default.

Gotcha: A private-looking path or undocumented endpoint is not an authentication or authorisation control.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.37 `09/registry-secrets` — Recover a secret from a model card

Category: infra-secrets · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `builtin/09/registry-secrets` · **Defensive pattern:** `secrets`

4.37.1 Challenge description

Review how the built-in challenge models recover a secret from a model card, focusing on its infra-secrets trust boundary.

4.37.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

4.37.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

4.37.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Built-in flag verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.38 `10/asset-inventory` — Inventory the high-value AI assets

Category: tm-assets · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/10/asset-inventory` · **Defensive pattern:** `threat`

4.38.1 Challenge description

Review how the built-in challenge models inventory the high-value ai assets, focusing on its tm-assets trust boundary.

4.38.2 Educational focus

A useful threat model explains relationships and complete paths rather than listing security keywords.

Trust boundary: Assets, principals, data flows, and effects cross one or more explicitly modelled trust boundaries.

4.38.3 Guided defensive review

1. Which asset and principal are involved?
2. What ordered path connects entry, privilege, action, and impact?

4.38.4 Defensive resolution

Record assets, trust boundaries, data flows, assumptions, ranked paths, owners, and controls tied to each boundary.

Validation: Review models for concrete actors and ordered relationships; reject disconnected keyword inventories.

Gotcha: A complete list of terms can still omit the relationship that makes a scenario possible.

Evidence status: Built-in artifact verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.39 `10/trust-boundaries` — Map trust boundaries & data flows

Category: tm-boundaries · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `builtin/10/trust-boundaries` · **Defensive pattern:** `threat`

4.39.1 Challenge description

Review how the built-in challenge models map trust boundaries & data flows, focusing on its tm-boundaries trust boundary.

4.39.2 Educational focus

A useful threat model explains relationships and complete paths rather than listing security keywords.

Trust boundary: Assets, principals, data flows, and effects cross one or more explicitly modelled trust boundaries.

4.39.3 Guided defensive review

1. Which asset and principal are involved?
2. What ordered path connects entry, privilege, action, and impact?

4.39.4 Defensive resolution

Record assets, trust boundaries, data flows, assumptions, ranked paths, owners, and controls tied to each boundary.

Validation: Review models for concrete actors and ordered relationships; reject disconnected keyword inventories.

Gotcha: A complete list of terms can still omit the relationship that makes a scenario possible.

Evidence status: Built-in artifact verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.40 `10/attack-paths` — Enumerate end-to-end attack paths

Category: tm-paths · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/10/attack-paths` · **Defensive pattern:** `threat`

4.40.1 Challenge description

Review how the built-in challenge models enumerate end-to-end attack paths, focusing on its tm-paths trust boundary.

4.40.2 Educational focus

A useful threat model explains relationships and complete paths rather than listing security keywords.

Trust boundary: Assets, principals, data flows, and effects cross one or more explicitly modelled trust boundaries.

4.40.3 Guided defensive review

1. Which asset and principal are involved?
2. What ordered path connects entry, privilege, action, and impact?

4.40.4 Defensive resolution

Record assets, trust boundaries, data flows, assumptions, ranked paths, owners, and controls tied to each boundary.

Validation: Review models for concrete actors and ordered relationships; reject disconnected keyword inventories.

Gotcha: A complete list of terms can still omit the relationship that makes a scenario possible.

Evidence status: Built-in artifact verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.41 `11/initial-access` — Initial access via indirect injection

Category: chain-access · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `builtin/11/initial-access` · **Defensive pattern:** `chain`

4.41.1 Challenge description

Review how the built-in challenge models initial access via indirect injection, focusing on its chain-access trust boundary.

4.41.2 Educational focus

Security review must follow end-to-end authority and data flow because local controls can fail in composition.

Trust boundary: Several individually limited weaknesses compose across content, identity, tools, data, and infrastructure.

4.41.3 Guided defensive review

1. Which earlier weakness enables the next step?
2. Which control breaks the chain earliest and most reliably?

4.41.4 Defensive resolution

Use layered controls, trace provenance, minimise ambient authority, segment systems, and retain cross-service evidence.

Validation: Validate each boundary independently with safe fixtures, then review the composed data flow without executing an attack chain.

Gotcha: A control may stop one step while leaving another route to the same privileged effect.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.42 `11/pivot-vault` — Pivot through ambient authority to the vault

Category: chain-pivot · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/11/pivot-vault` · **Defensive pattern:** `chain`

4.42.1 Challenge description

Review how the built-in challenge models pivot through ambient authority to the vault, focusing on its chain-pivot trust boundary.

4.42.2 Educational focus

Security review must follow end-to-end authority and data flow because local controls can fail in composition.

Trust boundary: Several individually limited weaknesses compose across content, identity, tools, data, and infrastructure.

4.42.3 Guided defensive review

1. Which earlier weakness enables the next step?
2. Which control breaks the chain earliest and most reliably?

4.42.4 Defensive resolution

Use layered controls, trace provenance, minimise ambient authority, segment systems, and retain cross-service evidence.

Validation: Validate each boundary independently with safe fixtures, then review the composed data flow without executing an attack chain.

Gotcha: A control may stop one step while leaving another route to the same privileged effect.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

4.43 `11/exfil-crown-jewel` — Exfiltrate the crown-jewel secret

Category: chain-exfil · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `builtin/11/exfil-crown-jewel` · **Defensive pattern:** `chain`

4.43.1 Challenge description

Review how the built-in challenge models exfiltrate the crown-jewel secret, focusing on its chain-exfil trust boundary.

4.43.2 Educational focus

Security review must follow end-to-end authority and data flow because local controls can fail in composition.

Trust boundary: Several individually limited weaknesses compose across content, identity, tools, data, and infrastructure.

4.43.3 Guided defensive review

1. Which earlier weakness enables the next step?
2. Which control breaks the chain earliest and most reliably?

4.43.4 Defensive resolution

Use layered controls, trace provenance, minimise ambient authority, segment systems, and retain cross-service evidence.

Validation: Validate each boundary independently with safe fixtures, then review the composed data flow without executing an attack chain.

Gotcha: A control may stop one step while leaving another route to the same privileged effect.

Evidence status: Built-in oracle verifier. Offensive solution: **not verified—outside the current scope**.

Provenance: RedSec module manifest and lesson

5 dvaa — Damn Vulnerable AI Agent

5.1 L1-01 — Extract the System Prompt

Category: data-exfiltration · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `dvaa/L1-01` · **Defensive pattern:** `secrets`

5.1.1 Challenge description

Review the Extract the System Prompt scenario as implemented by OpenA2A, focusing on its data-exfiltration boundary.

5.1.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

5.1.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

5.1.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.2 `L1-02` — API Key Leak

Category: data-exfiltration · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `dvaa/L1-02` · **Defensive pattern:** `secrets`

5.2.1 Challenge description

Review the API Key Leak scenario as implemented by OpenA2A, focusing on its data-exfiltration boundary.

5.2.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

5.2.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

5.2.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.3 `L1-03` — Basic Prompt Injection

Category: prompt-injection · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `dvaa/L1-03` · **Defensive pattern:** `prompt`

5.3.1 Challenge description

Review the Basic Prompt Injection scenario as implemented by OpenA2A, focusing on its prompt-injection boundary.

5.3.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

5.3.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

5.3.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.4 `L2-01` — Jailbreak via Roleplay

Category: jailbreak · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvaa/L2-01` · **Defensive pattern:** `prompt`

5.4.1 Challenge description

Review the Jailbreak via Roleplay scenario as implemented by OpenA2A, focusing on its jailbreak boundary.

5.4.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

5.4.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

5.4.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.5 `L2-02` — Context Window Manipulation

Category: context-manipulation · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvaa/L2-02` · **Defensive pattern:** `prompt`

5.5.1 Challenge description

Review the Context Window Manipulation scenario as implemented by OpenA2A, focusing on its context-manipulation boundary.

5.5.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

5.5.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

5.5.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.6 `L2-03` — MCP Path Traversal

Category: mcp-exploitation · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvaa/L2-03` · **Defensive pattern:** `path`

5.6.1 Challenge description

Review the MCP Path Traversal scenario as implemented by OpenA2A, focusing on its mcp-exploitation boundary.

5.6.2 Educational focus

Access decisions must use the final canonical target and remain safe across encoding, absolute paths, and symbolic links.

Trust boundary: An untrusted path or URI crosses into filesystem resolution outside an authorised root.

5.6.3 Guided defensive review

1. What is the canonical allowed root?
2. Can decoding, symlinks, alternate separators, or races change the resolved target?

5.6.4 Defensive resolution

Resolve paths safely, reject absolute and escaping targets, enforce containment after canonicalisation, and run with minimal filesystem access.

Validation: Use temporary in-root and out-of-root fixtures with encoded, absolute, separator, and symlink cases and assert fail-closed behavior.

Gotcha: String-prefix checks can approve sibling paths or links whose final target lies outside the intended root.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.7 `L2-04` — Persistent Memory Injection

Category: memory-injection · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvaa/L2-04` · **Defensive pattern:** `prompt`

5.7.1 Challenge description

Review the Persistent Memory Injection scenario as implemented by OpenA2A, focusing on its memory-injection boundary.

5.7.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

5.7.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

5.7.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.8 `L2-05` — Memory Credential Extraction

Category: memory-injection · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvaa/L2-05` · **Defensive pattern:** `secrets`

5.8.1 Challenge description

Review the Memory Credential Extraction scenario as implemented by OpenA2A, focusing on its memory-injection boundary.

5.8.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

5.8.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

5.8.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.9 `L2-06` — Context Padding Attack

Category: context-overflow · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvaa/L2-06` · **Defensive pattern:** `prompt`

5.9.1 Challenge description

Review the Context Padding Attack scenario as implemented by OpenA2A, focusing on its context-overflow boundary.

5.9.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

5.9.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

5.9.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.10 L2-07 — Safety Instruction Displacement

Category: context-overflow · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvaa/L2-07` · **Defensive pattern:** `prompt`

5.10.1 Challenge description

Review the Safety Instruction Displacement scenario as implemented by OpenA2A, focusing on its context-overflow boundary.

5.10.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

5.10.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

5.10.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.11 L2-08 — Malicious Tool Registration

Category: tool-registry-poisoning · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvaa/L2-08` · **Defensive pattern:** `tool-registration`

5.11.1 Challenge description

Review the Malicious Tool Registration scenario as implemented by OpenA2A, focusing on its tool-registry-poisoning boundary.

5.11.2 Educational focus

Registration is a software provenance decision: only authenticated publishers and reviewed, immutable tool definitions may become callable.

Trust boundary: A tool manifest crosses from an external registry or publisher into the trusted capability catalog.

5.11.3 Guided defensive review

1. Which registry and publisher identity are trusted?
2. Are manifest signatures, source revisions, and requested capabilities verified before approval?

5.11.4 Defensive resolution

Use approved registries, signed manifests, verified publisher identities, pinned source revisions, capability review, and an explicit tool allow-list.

Validation: Submit benign signed, unsigned, altered, and unapproved manifests and assert that only the approved identity and exact digest can register.

Gotcha: Schema-valid metadata can still describe an untrusted implementation; validation does not establish publisher or source authenticity.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.12 `L2-09` — Tool Call MITM

Category: tool-mitm · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvaa/L2-09` · **Defensive pattern:** `tool-mitm`

5.12.1 Challenge description

Review the Tool Call MITM scenario as implemented by OpenA2A, focusing on its tool-mitm boundary.

5.12.2 Educational focus

The host must authenticate the tool endpoint and bind each response to the intended request and server identity.

Trust boundary: Tool calls and results cross a network or intermediary that could replace the server, request, or response.

5.12.3 Guided defensive review

1. How is the server authenticated?
2. Are request identity, response identity, integrity, freshness, and channel binding verified?

5.12.4 Defensive resolution

Use authenticated encrypted transport, pinned service identity where appropriate, request-response correlation, replay protection, and integrity checks.

Validation: Use a benign test endpoint with the wrong identity or correlation value and assert the host rejects its responses.

Gotcha: TLS without correct hostname, trust-store, redirect, and response binding checks can still connect to the wrong tool.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.13 `L3-01` — Chained Prompt Injection

Category: prompt-injection · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `dvaa/L3-01` · **Defensive pattern:** `prompt`

5.13.1 Challenge description

Review the Chained Prompt Injection scenario as implemented by OpenA2A, focusing on its prompt-injection boundary.

5.13.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

5.13.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

5.13.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.14 L3-02 — SSRF via MCP

Category: mcp-exploitation · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `dvaa/L3-02` · **Defensive pattern:** `infra`

5.14.1 Challenge description

Review the SSRF via MCP scenario as implemented by OpenA2A, focusing on its mcp-exploitation boundary.

5.14.2 Educational focus

Workload identity and egress are authority boundaries; model-facing services should carry minimal ambient privilege.

Trust boundary: A model-facing workload crosses into cloud metadata, administrative APIs, registries, or network destinations.

5.14.3 Guided defensive review

1. Which destinations can the workload reach?
2. Which administrative or registry actions require authentication?

5.14.4 Defensive resolution

Default-deny egress, use workload identity, require admin authentication, isolate metadata, and keep secrets out of artifacts.

Validation: Review policy and use inert internal endpoints to confirm blocked destinations and authenticated administration.

Gotcha: Application input validation cannot compensate for broad network reach and over-privileged workload credentials.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.15 `L3-03` — Self-Replicating Memory Entry

Category: memory-injection · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `dvaa/L3-03` · **Defensive pattern:** `prompt`

5.15.1 Challenge description

Review the Self-Replicating Memory Entry scenario as implemented by OpenA2A, focusing on its memory-injection boundary.

5.15.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

5.15.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

5.15.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.16 `L3-04` — System Prompt Extraction via Context Pressure

Category: context-overflow · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `dvaa/L3-04` · **Defensive pattern:** `secrets`

5.16.1 Challenge description

Review the System Prompt Extraction via Context Pressure scenario as implemented by OpenA2A, focusing on its context-overflow boundary.

5.16.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

5.16.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

5.16.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.17 `L3-05` — Tool Typosquatting

Category: tool-registry-poisoning · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `dvaa/L3-05` · **Defensive pattern:** `tool-shadowing`

5.17.1 Challenge description

Review the Tool Typosquatting scenario as implemented by OpenA2A, focusing on its tool-registry-poisoning boundary.

5.17.2 Educational focus

Human-readable names are not stable identities; resolution must bind approval to one authenticated implementation.

Trust boundary: A tool name or namespace crosses from discovery into registry resolution and invocation.

5.17.3 Guided defensive review

1. What cryptographic identity owns this tool?
2. Can another registry entry, namespace, or later definition win resolution?

5.17.4 Defensive resolution

Use an immutable registry, signed and version-pinned definitions, unique namespaces, and approval bound to the server and definition digest.

Validation: Register benign lookalike and duplicate names and assert that ambiguous, unsigned, or changed identities fail closed.

Gotcha: Name allow-lists still fail when resolution can select a different implementation with the same or similar name.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.18 `L3-06` — Tool Chain Data Exfiltration

Category: tool-registry-poisoning · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `dvaa/L3-06` · **Defensive pattern:** `tool-dataflow`

5.18.1 Challenge description

Review the Tool Chain Data Exfiltration scenario as implemented by OpenA2A, focusing on its tool-registry-poisoning boundary.

5.18.2 Educational focus

Authorisation must follow provenance and data flow across the whole tool chain, not only each isolated call.

Trust boundary: Sensitive data crosses several individually authorised tool calls before reaching a new destination.

5.18.3 Guided defensive review

1. Which earlier result becomes a later argument?
2. Is the final destination permitted for that data classification?

5.18.4 Defensive resolution

Track taint and provenance across calls, apply destination-aware policy, minimise returned data, and require confirmation for cross-domain transfer.

Validation: Use synthetic labelled values to verify that allowed local processing cannot be composed into an unauthorised transfer.

Gotcha: Every call can look legitimate in isolation while their composition violates the original data-use purpose.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.19 L3-07 — Tool Shadowing

Category: tool-mitm · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `dvaa/L3-07` · **Defensive pattern:** `tool-shadowing`

5.19.1 Challenge description

Review the Tool Shadowing scenario as implemented by OpenA2A, focusing on its tool-mitm boundary.

5.19.2 Educational focus

Human-readable names are not stable identities; resolution must bind approval to one authenticated implementation.

Trust boundary: A tool name or namespace crosses from discovery into registry resolution and invocation.

5.19.3 Guided defensive review

1. What cryptographic identity owns this tool?
2. Can another registry entry, namespace, or later definition win resolution?

5.19.4 Defensive resolution

Use an immutable registry, signed and version-pinned definitions, unique namespaces, and approval bound to the server and definition digest.

Validation: Register benign lookalike and duplicate names and assert that ambiguous, unsigned, or changed identities fail closed.

Gotcha: Name allow-lists still fail when resolution can select a different implementation with the same or similar name.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.20 L3-08 — Traffic Redirection Attack

Category: tool-mitm · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `dvaa/L3-08` · **Defensive pattern:** `tool-redirect`

5.20.1 Challenge description

Review the Traffic Redirection Attack scenario as implemented by OpenA2A, focusing on its tool-mitm boundary.

5.20.2 Educational focus

Every hop and resolved destination must remain within the approved endpoint policy and authenticated service identity.

Trust boundary: A configured tool endpoint crosses DNS, redirect, proxy, and routing boundaries before reaching the final service.

5.20.3 Guided defensive review

1. Which redirects and resolved addresses are allowed?
2. Does authentication bind to the final destination after DNS and proxy routing?

5.20.4 Defensive resolution

Pin approved schemes and hosts, restrict redirects, validate resolved addresses, authenticate the final service, and re-check policy on every hop.

Validation: Use inert redirect and alternate-resolution fixtures and assert the client refuses destinations outside the approved set.

Gotcha: Validating only the original URL misses a later redirect or resolution to a different trust zone.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.21 `L4-01` — Compromise SecureBot

Category: mixed · **Difficulty:** expert · **Declared variants:** 1 **Catalog key:** `dvaa/L4-01` · **Defensive pattern:** `chain`

5.21.1 Challenge description

Review the Compromise SecureBot scenario as implemented by OpenA2A, focusing on its mixed boundary.

5.21.2 Educational focus

Security review must follow end-to-end authority and data flow because local controls can fail in composition.

Trust boundary: Several individually limited weaknesses compose across content, identity, tools, data, and infrastructure.

5.21.3 Guided defensive review

1. Which earlier weakness enables the next step?
2. Which control breaks the chain earliest and most reliably?

5.21.4 Defensive resolution

Use layered controls, trace provenance, minimise ambient authority, segment systems, and retain cross-service evidence.

Validation: Validate each boundary independently with safe fixtures, then review the composed data flow without executing an attack chain.

Gotcha: A control may stop one step while leaving another route to the same privileged effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

5.22 L4-02 — Agent-to-Agent Attack Chain

Category: agent-to-agent · **Difficulty:** expert · **Declared variants:** 1 **Catalog key:** `dvaa/L4-02` · **Defensive pattern:** `chain`

5.22.1 Challenge description

Review the Agent-to-Agent Attack Chain scenario as implemented by OpenA2A, focusing on its agent-to-agent boundary.

5.22.2 Educational focus

Security review must follow end-to-end authority and data flow because local controls can fail in composition.

Trust boundary: Several individually limited weaknesses compose across content, identity, tools, data, and infrastructure.

5.22.3 Guided defensive review

1. Which earlier weakness enables the next step?
2. Which control breaks the chain earliest and most reliably?

5.22.4 Defensive resolution

Use layered controls, trace provenance, minimise ambient authority, segment systems, and retain cross-service evidence.

Validation: Validate each boundary independently with safe fixtures, then review the composed data flow without executing an attack chain.

Gotcha: A control may stop one step while leaving another route to the same privileged effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/opena2a-org/damn-vulnerable-ai-agent/blob/faf3fb172ef526d12db0eb5377701549693118a6/src/challenges/index.js

6 `mcpgoat` — MCPGoat

6.1 `A1` — Tool Poisoning

Category: MCP-specific · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/A1` · **Defensive pattern:** `tool-poisoning`

6.1.1 Challenge description

Review the Tool Poisoning scenario as implemented by Sabyasachi Dhal, focusing on its MCP-specific boundary.

6.1.2 Educational focus

Tool metadata is untrusted content and must not silently grant authority or inject instructions.

Trust boundary: Tool descriptions and results cross from a server-controlled metadata plane into model planning and user approval.

6.1.3 Guided defensive review

1. Which metadata reaches the model or approval UI?
2. Does execution use the same reviewed, authenticated definition?

6.1.4 Defensive resolution

Authenticate servers, sanitise and canonicalise metadata, show security-relevant fields, and authorise every call outside the description.

Validation: Use benign instruction-like metadata to confirm it remains visible as data and cannot broaden a tool's permitted effect.

Gotcha: A harmless tool name can conceal instructions in descriptions, annotations, or returned content.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.2 A2 — Tool Shadowing / Trusted-Tool Override

Category: MCP-specific · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/A2` · **Defensive pattern:** `tool-shadowing`

6.2.1 Challenge description

Review the Tool Shadowing / Trusted-Tool Override scenario as implemented by Sabyasachi Dhal, focusing on its MCP-specific boundary.

6.2.2 Educational focus

Human-readable names are not stable identities; resolution must bind approval to one authenticated implementation.

Trust boundary: A tool name or namespace crosses from discovery into registry resolution and invocation.

6.2.3 Guided defensive review

1. What cryptographic identity owns this tool?
2. Can another registry entry, namespace, or later definition win resolution?

6.2.4 Defensive resolution

Use an immutable registry, signed and version-pinned definitions, unique namespaces, and approval bound to the server and definition digest.

Validation: Register benign lookalike and duplicate names and assert that ambiguous, unsigned, or changed identities fail closed.

Gotcha: Name allow-lists still fail when resolution can select a different implementation with the same or similar name.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.3 A3 — Rug Pull / Tool Mutation (TOCTOU)

Category: MCP-specific · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/A3` · **Defensive pattern:** `rug-pull`

6.3.1 Challenge description

Review the Rug Pull / Tool Mutation (TOCTOU) scenario as implemented by Sabyasachi Dhal, focusing on its MCP-specific boundary.

6.3.2 Educational focus

Approval applies to an immutable definition digest and must be invalidated when security-relevant metadata or behavior changes.

Trust boundary: A previously reviewed tool definition changes between approval, discovery, and execution.

6.3.3 Guided defensive review

1. Which digest was approved?
2. What forces re-review when the definition or server identity changes?

6.3.4 Defensive resolution

Pin signed definitions, compare canonical digests before every call, cache immutably, and require re-approval for any change.

Validation: Use two benign versions of one tool and assert that the second cannot run under approval granted to the first.

Gotcha: Periodic refresh without digest-bound approval creates a time-of-check/time-of-use gap.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.4 B1 — Indirect Prompt Injection via Tool Output

Category: Prompt/Context · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/B1` · **Defensive pattern:** `prompt`

6.4.1 Challenge description

Review the Indirect Prompt Injection via Tool Output scenario as implemented by Sabyasachi Dhal, focusing on its Prompt/Context boundary.

6.4.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

6.4.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

6.4.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.5 D1 — Command Injection (RCE)

Category: Injection sink · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/D1` · **Defensive pattern:** `command`

6.5.1 Challenge description

Review the Command Injection (RCE) scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

6.5.2 Educational focus

Data must remain structured arguments and must never become executable command syntax.

Trust boundary: Untrusted arguments cross into a shell, interpreter, dynamic loader, or operating-system process boundary.

6.5.3 Guided defensive review

1. Is a shell or evaluator involved?
2. Are executable identity and every argument fixed or independently allow-listed?

6.5.4 Defensive resolution

Remove shell evaluation, call fixed executables with argument arrays, validate typed values, drop privileges, and sandbox the process.

Validation: Use harmless metacharacter and unexpected-argument fixtures and assert they are rejected or passed literally without side effects.

Gotcha: Escaping for one shell or platform is fragile; avoiding the interpreter is the durable control.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.6 `D2` — Path Traversal / Arbitrary File Read

Category: Injection sink · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/D2` · **Defensive pattern:** `path`

6.6.1 Challenge description

Review the Path Traversal / Arbitrary File Read scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

6.6.2 Educational focus

Access decisions must use the final canonical target and remain safe across encoding, absolute paths, and symbolic links.

Trust boundary: An untrusted path or URI crosses into filesystem resolution outside an authorised root.

6.6.3 Guided defensive review

1. What is the canonical allowed root?
2. Can decoding, symlinks, alternate separators, or races change the resolved target?

6.6.4 Defensive resolution

Resolve paths safely, reject absolute and escaping targets, enforce containment after canonicalisation, and run with minimal filesystem access.

Validation: Use temporary in-root and out-of-root fixtures with encoded, absolute, separator, and symlink cases and assert fail-closed behavior.

Gotcha: String-prefix checks can approve sibling paths or links whose final target lies outside the intended root.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.7 D3 — Server-Side Request Forgery

Category: Injection sink · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/D3` · **Defensive pattern:** `infra`

6.7.1 Challenge description

Review the Server-Side Request Forgery scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

6.7.2 Educational focus

Workload identity and egress are authority boundaries; model-facing services should carry minimal ambient privilege.

Trust boundary: A model-facing workload crosses into cloud metadata, administrative APIs, registries, or network destinations.

6.7.3 Guided defensive review

1. Which destinations can the workload reach?
2. Which administrative or registry actions require authentication?

6.7.4 Defensive resolution

Default-deny egress, use workload identity, require admin authentication, isolate metadata, and keep secrets out of artifacts.

Validation: Review policy and use inert internal endpoints to confirm blocked destinations and authenticated administration.

Gotcha: Application input validation cannot compensate for broad network reach and over-privileged workload credentials.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.8 D4 — SQL Injection

Category: Injection sink · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/D4` · **Defensive pattern:** `sql`

6.8.1 Challenge description

Review the SQL Injection scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

6.8.2 Educational focus

Queries must keep code and data separate, including identifiers, sorting, and dynamically selected clauses.

Trust boundary: Untrusted values cross into relational query structure and database privileges.

6.8.3 Guided defensive review

1. Which query elements are parameterised?
2. Does the database account have only the permissions required by this operation?

6.8.4 Defensive resolution

Use parameterised queries, allow-list non-parameterisable identifiers, constrain result size, and grant least-privilege database roles.

Validation: Use inert quote, wildcard, comment, and type-mismatch fixtures and assert query shape and authorised results remain unchanged.

Gotcha: Parameters protect values but do not make attacker-selected table names, operators, or sort clauses safe.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.9 `C2` — Broken Authorization / Confused Deputy

Category: AuthN/AuthZ · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/C2` · **Defensive pattern:** `identity`

6.9.1 Challenge description

Review the Broken Authorization / Confused Deputy scenario as implemented by Sabyasachi Dhal, focusing on its AuthN/AuthZ boundary.

6.9.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

6.9.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

6.9.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.10 C3 — Insecure Direct Object Reference

Category: AuthN/AuthZ · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/C3` · **Defensive pattern:** `identity`

6.10.1 Challenge description

Review the Insecure Direct Object Reference scenario as implemented by Sabyasachi Dhal, focusing on its AuthN/AuthZ boundary.

6.10.2 Educational focus

Declared identity and delegated context are claims; authenticate the caller and authorise the resulting action.

Trust boundary: A principal, object, task, or delegated action crosses an identity and authorisation boundary.

6.10.3 Guided defensive review

1. Which principal is authenticated?
2. Is the object and delegated action authorised for that principal?

6.10.4 Defensive resolution

Use signed identities, audience-bound credentials, object-level checks, task ownership, and least privilege.

Validation: Use negative authorisation tests for wrong users, objects, audiences, tasks, and replayed context identifiers.

Gotcha: A valid sender or token does not automatically authorise every downstream object or action.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.11 E1 — Sensitive Data Exposure

Category: Secrets/Exposure · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/E1` · **Defensive pattern:** `secrets`

6.11.1 Challenge description

Review the Sensitive Data Exposure scenario as implemented by Sabyasachi Dhal, focusing on its Secrets/Exposure boundary.

6.11.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

6.11.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

6.11.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.12 A9 — Invisible-Text Tool Poisoning

Category: MCP-specific · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/A9` · **Defensive pattern:** `unicode`

6.12.1 Challenge description

Review the Invisible-Text Tool Poisoning scenario as implemented by Sabyasachi Dhal, focusing on its MCP-specific boundary.

6.12.2 Educational focus

Security review and execution must use one canonical representation while preserving warnings about concealed characters.

Trust boundary: Unicode formatting and confusable characters cross from raw metadata into scanning, display, and model-visible text.

6.12.3 Guided defensive review

1. Which normalization form is enforced?
2. Are bidirectional controls, zero-width characters, and confusables exposed to reviewers?

6.12.4 Defensive resolution

Normalize consistently, reject or visibly escape control characters, detect confusables, and scan the same canonical text shown to users.

Validation: Use harmless strings containing bidi, zero-width, and confusable characters and assert identical scanner and display decisions.

Gotcha: Stripping a subset of invisible characters can create a second, differently interpreted representation.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.13 B2 — Indirect Injection via Resource Content

Category: Prompt/Context · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/B2` · **Defensive pattern:** `prompt`

6.13.1 Challenge description

Review the Indirect Injection via Resource Content scenario as implemented by Sabyasachi Dhal, focusing on its Prompt/Context boundary.

6.13.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

6.13.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

6.13.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.14 B3 — Prompt-Template Injection

Category: Prompt/Context · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/B3` · **Defensive pattern:** `prompt`

6.14.1 Challenge description

Review the Prompt-Template Injection scenario as implemented by Sabyasachi Dhal, focusing on its Prompt/Context boundary.

6.14.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

6.14.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

6.14.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.15 B5 — Sampling Abuse (server-driven LLM calls)

Category: MCP-specific · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/B5` · **Defensive pattern:** `sampling`

6.15.1 Challenge description

Review the Sampling Abuse (server-driven LLM calls) scenario as implemented by Sabyasachi Dhal, focusing on its MCP-specific boundary.

6.15.2 Educational focus

Sampling is a privileged host action that needs explicit purpose, context minimisation, consent, and resource limits.

Trust boundary: A server-requested model call crosses back into host context, credentials, tokens, and compute budgets.

6.15.3 Guided defensive review

1. Which context is shared with the sampled model?
2. Who authorises the purpose, model, token budget, and returned destination?

6.15.4 Defensive resolution

Require informed consent, minimise context, isolate credentials, allow-list models and purposes, cap tokens and cost, and audit results.

Validation: Use synthetic host context and bounded budgets to confirm an untrusted server cannot expand context or repeat calls silently.

Gotcha: Approving the parent tool call does not automatically approve every server-initiated model call it requests.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.16 C4 — OAuth Token-Audience Confusion

Category: AuthN/AuthZ · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/C4` · **Defensive pattern:** `oauth-audience`

6.16.1 Challenge description

Review the OAuth Token-Audience Confusion scenario as implemented by Sabyasachi Dhal, focusing on its AuthN/AuthZ boundary.

6.16.2 Educational focus

Token validity is insufficient unless issuer, audience, scope, client, subject, and delegation all match the requested operation.

Trust boundary: An OAuth token crosses from its intended resource server or client into a different audience or delegated action.

6.16.3 Guided defensive review

1. Which exact resource audience issued the token?
2. Are scope and delegated subject valid for this tool and action?

6.16.4 Defensive resolution

Validate issuer and exact audience, use resource-specific tokens, restrict scopes, bind delegation, and reject token forwarding across services.

Validation: Use synthetic tokens with correct signatures but wrong audiences, clients, scopes, or subjects and assert rejection.

Gotcha: A token accepted by one service must not become a bearer credential for every service in the agent chain.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.17 D6 — Server-Side Template Injection

Category: Injection sink · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/D6` · **Defensive pattern:** `template`

6.17.1 Challenge description

Review the Server-Side Template Injection scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

6.17.2 Educational focus

Users may supply data for a fixed template but must not define executable template structure.

Trust boundary: Untrusted text crosses into template source, expressions, object traversal, or server-side helpers.

6.17.3 Guided defensive review

1. Can input become template source or an expression?
2. Which objects, filters, and helpers are reachable during rendering?

6.17.4 Defensive resolution

Use fixed logic-light templates, pass typed data, enable sandboxing and autoescape, and remove dangerous helpers and object access.

Validation: Render harmless expression-like fixtures and assert they remain literal text with no object access or evaluation.

Gotcha: Output escaping prevents markup injection but does not stop server-side expression evaluation.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.18 F1 — DNS Rebinding / Missing Origin Validation

Category: Transport · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/F1` · **Defensive pattern:** `dns-rebinding`

6.18.1 Challenge description

Review the DNS Rebinding / Missing Origin Validation scenario as implemented by Sabyasachi Dhal, focusing on its Transport boundary.

6.18.2 Educational focus

Origin checks and destination policy must bind to stable identities and validate every resolved connection target.

Trust boundary: A hostname crosses browser-origin and DNS resolution boundaries and may resolve to a different address over time.

6.18.3 Guided defensive review

1. Is the Origin validated exactly?
2. Are all resolved addresses checked at connection time and after DNS changes?

6.18.4 Defensive resolution

Require exact origins, re-resolve and validate destination addresses, reject private or disallowed ranges, and avoid trusting hostname strings alone.

Validation: Use controlled DNS fixtures that change between safe addresses and assert disallowed resolutions never receive a connection.

Gotcha: A hostname that was safe during validation can resolve differently when the connection is opened.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.19 F2 — CORS Misconfiguration

Category: Transport · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/F2` · **Defensive pattern:** `cors`

6.19.1 Challenge description

Review the CORS Misconfiguration scenario as implemented by Sabyasachi Dhal, focusing on its Transport boundary.

6.19.2 Educational focus

Cross-origin access must use a narrow exact allow-list and must never reflect arbitrary origins with credentials.

Trust boundary: Browser credentials and responses cross from an MCP HTTP service into a requesting web origin.

6.19.3 Guided defensive review

1. Which exact origins need access?
2. Are credentials, preflight methods, headers, and cached decisions constrained consistently?

6.19.4 Defensive resolution

Allow-list exact HTTPS origins, minimise methods and headers, vary caches by Origin, and disable credentials unless strictly required.

Validation: Send harmless preflights from allowed, sibling, null, and attacker-controlled origins and assert only the exact policy succeeds.

Gotcha: Wildcard subdomains and reflected origins can turn an apparently narrow policy into credentialed cross-origin access.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.20 C6 — Predictable Session IDs (hijack)

Category: Transport · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/C6` · **Defensive pattern:** `session-security`

6.20.1 Challenge description

Review the Predictable Session IDs (hijack) scenario as implemented by Sabyasachi Dhal, focusing on its Transport boundary.

6.20.2 Educational focus

Session identifiers must be unpredictable, confidential, short-lived, and bound to the authenticated principal and channel context.

Trust boundary: A session identifier crosses clients, requests, reconnects, and server-side state ownership boundaries.

6.20.3 Guided defensive review

1. How much entropy and lifetime does the identifier have?
2. Can another principal replay, fix, or reuse it across connections?

6.20.4 Defensive resolution

Use cryptographic random identifiers, rotate on authentication changes, bind ownership server-side, expire promptly, and protect transport and storage.

Validation: Create multiple synthetic sessions and test replay, cross-user reuse, fixation, expiry, and logout invalidation.

Gotcha: A random-looking identifier still fails if it remains valid across users, privilege changes, or excessive time.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.21 G1 — Unbounded Consumption (cost/DoS)

Category: DoS/Cost · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/G1` · **Defensive pattern:** `dos`

6.21.1 Challenge description

Review the Unbounded Consumption (cost/DoS) scenario as implemented by Sabyasachi Dhal, focusing on its DoS/Cost boundary.

6.21.2 Educational focus

Availability and spend are security properties that require explicit per-request and aggregate budgets.

Trust boundary: Attacker-controlled work crosses compute, memory, token, concurrency, or financial budget boundaries.

6.21.3 Guided defensive review

1. Which resource grows with input?
2. Where are tenant, tool, and global budgets enforced?

6.21.4 Defensive resolution

Set timeouts, size and token limits, concurrency quotas, rate limits, cancellation, and cost telemetry.

Validation: Use bounded synthetic workloads to assert predictable rejection before resource use approaches operational limits.

Gotcha: Per-request limits alone miss distributed or repeated consumption across sessions and identities.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.22 G4 — Regular-Expression DoS (ReDoS)

Category: DoS/Cost · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/G4` · **Defensive pattern:** `redos`

6.22.1 Challenge description

Review the Regular-Expression DoS (ReDoS) scenario as implemented by Sabyasachi Dhal, focusing on its DoS/Cost boundary.

6.22.2 Educational focus

Pattern safety and bounded input must be established before regex matching occurs on a shared service thread.

Trust boundary: Attacker-controlled text crosses into a regular-expression engine whose backtracking cost can grow nonlinearly.

6.22.3 Guided defensive review

1. Can the pattern exhibit catastrophic backtracking?
2. What input, time, and concurrency limits bound each match?

6.22.4 Defensive resolution

Use linear-time patterns or engines, simplify ambiguous repetition, cap input length, set timeouts, and isolate expensive matching.

Validation: Run bounded synthetic worst-case strings in a timeout-controlled test and assert predictable rejection and latency.

Gotcha: A request rate limit does not prevent one accepted input from monopolising a worker for an excessive duration.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.23 D5 — NoSQL Injection

Category: Injection sink · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/D5` · **Defensive pattern:** `nosql`

6.23.1 Challenge description

Review the NoSQL Injection scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

6.23.2 Educational focus

Application data must not be interpreted as query operators or prototype-bearing objects.

Trust boundary: Untrusted JSON or object fields cross into NoSQL operators, selectors, projections, or update structure.

6.23.3 Guided defensive review

1. Which keys and value types are accepted?
2. Can nested operator keys or type confusion alter selector meaning?

6.23.4 Defensive resolution

Validate against a closed schema, construct queries from typed fields, reject operator-prefixed keys, and use least-privilege data access.

Validation: Use benign nested objects, arrays, unexpected types, and operator-like keys and assert the query builder rejects them.

Gotcha: Escaping strings does not help when the dangerous input is a structured operator object.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.24 D7 — XML External Entity (XXE)

Category: Injection sink · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/D7` · **Defensive pattern:** `xml`

6.24.1 Challenge description

Review the XML External Entity (XXE) scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

6.24.2 Educational focus

XML features that resolve external content must be disabled before parsing untrusted documents.

Trust boundary: Untrusted XML crosses into entity resolution, DTD processing, XInclude, and parser-controlled network or file access.

6.24.3 Guided defensive review

1. Are DTDs and external entities disabled?
2. Can the parser access files, networks, schemas, or includes?

6.24.4 Defensive resolution

Use a hardened parser with DTDs, external entities, XInclude, and network access disabled; enforce size and depth limits.

Validation: Parse inert documents containing disabled declarations and oversized nesting and assert deterministic rejection without I/O.

Gotcha: Blocking one entity syntax is insufficient when schemas, parameter entities, or includes still resolve externally.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.25 D8 — Insecure Deserialization (prototype pollution)

Category: Injection sink · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/D8` · **Defensive pattern:** `deserialization`

6.25.1 Challenge description

Review the Insecure Deserialization (prototype pollution) scenario as implemented by Sabyasachi Dhal, focusing on its Injection sink boundary.

6.25.2 Educational focus

Deserialization must produce schema-validated data objects without executable types or prototype mutation.

Trust boundary: Untrusted serialized objects cross into constructors, prototypes, type hooks, or recursive merge logic.

6.25.3 Guided defensive review

1. Which types and hooks can be instantiated?
2. Can special keys or merge behavior alter prototypes or application defaults?

6.25.4 Defensive resolution

Prefer plain data formats, validate a closed schema, reject dangerous keys and polymorphic types, and avoid recursive merges into prototypes.

Validation: Use inert unexpected types, deep objects, and prototype-like keys and assert the parser returns isolated plain data or rejects them.

Gotcha: A trusted transport does not make attacker-controlled object graphs or type metadata safe to instantiate.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

6.26 `H1` — Supply Chain (typosquat / unsigned package)

Category: Supply chain · **Difficulty:** tiered · **Declared variants:** 3 **Catalog key:** `mcpgoat/H1` · **Defensive pattern:** `dependency-confusion`

6.26.1 Challenge description

Review the Supply Chain (typosquat / unsigned package) scenario as implemented by Sabyasachi Dhal, focusing on its Supply chain boundary.

6.26.2 Educational focus

Dependency identity must bind namespace, registry, version, and digest rather than relying on the package name alone.

Trust boundary: A package name crosses internal and public registries where resolution priority may select an unintended publisher.

6.26.3 Guided defensive review

1. Which registry owns the namespace?
2. Can a public package or lookalike name outrank the intended internal dependency?

6.26.4 Defensive resolution

Use scoped private namespaces, explicit registries, lockfiles and hashes, signed provenance, and deny undeclared sources.

Validation: Resolve benign test packages against isolated registries and assert public, unsigned, or hash-mismatched candidates fail closed.

Gotcha: Pinning a version does not help when the same name and version can resolve from the wrong registry.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/SabyasachiDhal/MCPGoat/blob/80ceaf7ffedb8e0e74fd612798a6e09dfdd543ed/docs/SOLUTIONS.md

7 `dvmcp` — Damn Vulnerable MCP Server

7.1 `01` — Basic Prompt Injection

Category: Prompt/Context · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `dvmcp/01` · **Defensive pattern:** `prompt`

7.1.1 Challenge description

Review the Basic Prompt Injection scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its Prompt/Context boundary.

7.1.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

7.1.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

7.1.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

7.2 02 — Tool Poisoning

Category: MCP-specific · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `dvmcp/02` · **Defensive pattern:** `tool-poisoning`

7.2.1 Challenge description

Review the Tool Poisoning scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its MCP-specific boundary.

7.2.2 Educational focus

Tool metadata is untrusted content and must not silently grant authority or inject instructions.

Trust boundary: Tool descriptions and results cross from a server-controlled metadata plane into model planning and user approval.

7.2.3 Guided defensive review

1. Which metadata reaches the model or approval UI?
2. Does execution use the same reviewed, authenticated definition?

7.2.4 Defensive resolution

Authenticate servers, sanitise and canonicalise metadata, show security-relevant fields, and authorise every call outside the description.

Validation: Use benign instruction-like metadata to confirm it remains visible as data and cannot broaden a tool's permitted effect.

Gotcha: A harmless tool name can conceal instructions in descriptions, annotations, or returned content.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

7.3 03 — Excessive Permission Scope

Category: AuthN/AuthZ · **Difficulty:** easy · **Declared variants:** 1 **Catalog key:** `dvmcp/03` · **Defensive pattern:** `scope`

7.3.1 Challenge description

Review the Excessive Permission Scope scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its AuthN/AuthZ boundary.

7.3.2 Educational focus

Servers must enforce the intersection of declared scope, authenticated identity, and canonical resource location.

Trust boundary: A client-declared root, permission, or resource scope crosses into server-side access control.

7.3.3 Guided defensive review

1. Who defines and enforces the allowed scope?
2. Is the resource canonicalised before containment and permission checks?

7.3.4 Defensive resolution

Enforce scopes server-side, canonicalise resource identifiers, deny ambient access, and apply least privilege to the host process.

Validation: Use synthetic in-scope and out-of-scope resources, including encoded and absolute forms, to confirm containment fails closed.

Gotcha: Advertising a root or permission to the model does not enforce it at the filesystem or service boundary.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

7.4 04 — Rug Pull Attack

Category: MCP-specific · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvmcp/04` · **Defensive pattern:** `rug-pull`

7.4.1 Challenge description

Review the Rug Pull Attack scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its MCP-specific boundary.

7.4.2 Educational focus

Approval applies to an immutable definition digest and must be invalidated when security-relevant metadata or behavior changes.

Trust boundary: A previously reviewed tool definition changes between approval, discovery, and execution.

7.4.3 Guided defensive review

1. Which digest was approved?
2. What forces re-review when the definition or server identity changes?

7.4.4 Defensive resolution

Pin signed definitions, compare canonical digests before every call, cache immutably, and require re-approval for any change.

Validation: Use two benign versions of one tool and assert that the second cannot run under approval granted to the first.

Gotcha: Periodic refresh without digest-bound approval creates a time-of-check/time-of-use gap.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

7.5 05 — Tool Shadowing

Category: MCP-specific · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvmcp/05` · **Defensive pattern:** `tool-shadowing`

7.5.1 Challenge description

Review the Tool Shadowing scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its MCP-specific boundary.

7.5.2 Educational focus

Human-readable names are not stable identities; resolution must bind approval to one authenticated implementation.

Trust boundary: A tool name or namespace crosses from discovery into registry resolution and invocation.

7.5.3 Guided defensive review

1. What cryptographic identity owns this tool?
2. Can another registry entry, namespace, or later definition win resolution?

7.5.4 Defensive resolution

Use an immutable registry, signed and version-pinned definitions, unique namespaces, and approval bound to the server and definition digest.

Validation: Register benign lookalike and duplicate names and assert that ambiguous, unsigned, or changed identities fail closed.

Gotcha: Name allow-lists still fail when resolution can select a different implementation with the same or similar name.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

7.6 06 — Indirect Prompt Injection

Category: Prompt/Context · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvmcp/06` · **Defensive pattern:** `prompt`

7.6.1 Challenge description

Review the Indirect Prompt Injection scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its Prompt/Context boundary.

7.6.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

7.6.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

7.6.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

7.7 07 — Token Theft

Category: Secrets/Exposure · **Difficulty:** medium · **Declared variants:** 1 **Catalog key:** `dvmcp/07` · **Defensive pattern:** `secrets`

7.7.1 Challenge description

Review the Token Theft scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its Secrets/Exposure boundary.

7.7.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

7.7.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

7.7.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

7.8 `08` — Malicious Code Execution

Category: Injection sink · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `dvmcp/08` · **Defensive pattern:** `command`

7.8.1 Challenge description

Review the Malicious Code Execution scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its Injection sink boundary.

7.8.2 Educational focus

Data must remain structured arguments and must never become executable command syntax.

Trust boundary: Untrusted arguments cross into a shell, interpreter, dynamic loader, or operating-system process boundary.

7.8.3 Guided defensive review

1. Is a shell or evaluator involved?
2. Are executable identity and every argument fixed or independently allow-listed?

7.8.4 Defensive resolution

Remove shell evaluation, call fixed executables with argument arrays, validate typed values, drop privileges, and sandbox the process.

Validation: Use harmless metacharacter and unexpected-argument fixtures and assert they are rejected or passed literally without side effects.

Gotcha: Escaping for one shell or platform is fragile; avoiding the interpreter is the durable control.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

7.9 09 — Remote Access Control

Category: Injection sink · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `dvmcp/09` · **Defensive pattern:** `command`

7.9.1 Challenge description

Review the Remote Access Control scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its Injection sink boundary.

7.9.2 Educational focus

Data must remain structured arguments and must never become executable command syntax.

Trust boundary: Untrusted arguments cross into a shell, interpreter, dynamic loader, or operating-system process boundary.

7.9.3 Guided defensive review

1. Is a shell or evaluator involved?
2. Are executable identity and every argument fixed or independently allow-listed?

7.9.4 Defensive resolution

Remove shell evaluation, call fixed executables with argument arrays, validate typed values, drop privileges, and sandbox the process.

Validation: Use harmless metacharacter and unexpected-argument fixtures and assert they are rejected or passed literally without side effects.

Gotcha: Escaping for one shell or platform is fragile; avoiding the interpreter is the durable control.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

7.10 10 — Multi-Vector Attack

Category: Attack chain · **Difficulty:** hard · **Declared variants:** 1 **Catalog key:** `dvmcp/10` · **Defensive pattern:** `chain`

7.10.1 Challenge description

Review the Multi-Vector Attack scenario as implemented by Harish Santhanalakshmi Ganesan, focusing on its Attack chain boundary.

7.10.2 Educational focus

Security review must follow end-to-end authority and data flow because local controls can fail in composition.

Trust boundary: Several individually limited weaknesses compose across content, identity, tools, data, and infrastructure.

7.10.3 Guided defensive review

1. Which earlier weakness enables the next step?
2. Which control breaks the chain earliest and most reliably?

7.10.4 Defensive resolution

Use layered controls, trace provenance, minimise ambient authority, segment systems, and retain cross-service evidence.

Validation: Validate each boundary independently with safe fixtures, then review the composed data flow without executing an attack chain.

Gotcha: A control may stop one step while leaving another route to the same privileged effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/harishsg993010/damn-vulnerable-MCP-server/tree/79734c19f5104cd11486c90926d245560f53befa/solutions

8 `appsecco-mcp` — Vulnerable MCP Servers Lab

8.1 `filesystem-workspace-actions` — Filesystem Workspace Actions

Category: Path traversal / code execution · **Difficulty:** ungraded · **Declared variants:** 1 **Catalog key:** `appsecco-mcp/filesystem-workspace-actions` · **Defensive pattern:** `filesystem-tools`

8.1.1 Challenge description

Review the Filesystem Workspace Actions scenario as implemented by Appsecco, focusing on its Path traversal / code execution boundary.

8.1.2 Educational focus

File access and action execution need separate allow-lists enforced after canonical path resolution.

Trust boundary: Workspace paths and requested operations cross into filesystem and process capabilities on the host.

8.1.3 Guided defensive review

1. Which roots and operations are authorised independently?
2. Can file content or names become commands, scripts, or executable arguments?

8.1.4 Defensive resolution

Canonicalise and contain paths, expose narrow operation-specific APIs, avoid shell interpretation, and isolate the worker account and filesystem.

Validation: Use disposable files and no-op actions to test traversal, symlinks, unsupported operations, and command-like names without executing payloads.

Gotcha: A path can remain in the workspace while the selected operation still creates an unintended executable effect.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

8.2 `indirect-prompt-injection` — Indirect Prompt Injection (stdio)

Category: Prompt/Context · **Difficulty:** ungraded · **Declared variants:** 1 **Catalog key:** `appsecco-mcp/indirect-prompt-injection` · **Defensive pattern:** `prompt`

8.2.1 Challenge description

Review the Indirect Prompt Injection (stdio) scenario as implemented by Appsecco, focusing on its Prompt/Context boundary.

8.2.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

8.2.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

8.2.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

8.3 `indirect-prompt-injection-remote` — Indirect Prompt Injection (HTTP+SSE)

Category: Prompt/Context · **Difficulty:** ungraded · **Declared variants:** 1 **Catalog key:** `appsecco-mcp/indirect-prompt-injection-remote` · **Defensive pattern:** `prompt`

8.3.1 Challenge description

Review the Indirect Prompt Injection (HTTP+SSE) scenario as implemented by Appsecco, focusing on its Prompt/Context boundary.

8.3.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

8.3.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

8.3.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

8.4 `malicious-code-exec` — Malicious Code Execution

Category: Injection sink · **Difficulty:** ungraded · **Declared variants:** 1 **Catalog key:** `appsecco-mcp/malicious-code-exec` · **Defensive pattern:** `command`

8.4.1 Challenge description

Review the Malicious Code Execution scenario as implemented by Appsecco, focusing on its Injection sink boundary.

8.4.2 Educational focus

Data must remain structured arguments and must never become executable command syntax.

Trust boundary: Untrusted arguments cross into a shell, interpreter, dynamic loader, or operating-system process boundary.

8.4.3 Guided defensive review

1. Is a shell or evaluator involved?
2. Are executable identity and every argument fixed or independently allow-listed?

8.4.4 Defensive resolution

Remove shell evaluation, call fixed executables with argument arrays, validate typed values, drop privileges, and sandbox the process.

Validation: Use harmless metacharacter and unexpected-argument fixtures and assert they are rejected or passed literally without side effects.

Gotcha: Escaping for one shell or platform is fragile; avoiding the interpreter is the durable control.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

8.5 `malicious-tools` — Malicious Tools

Category: Tool output poisoning · **Difficulty:** ungraded · **Declared variants:** 1 **Catalog key:** `appsecco-mcp/malicious-tools` · **Defensive pattern:** `tool-poisoning`

8.5.1 Challenge description

Review the Malicious Tools scenario as implemented by Appsecco, focusing on its Tool output poisoning boundary.

8.5.2 Educational focus

Tool metadata is untrusted content and must not silently grant authority or inject instructions.

Trust boundary: Tool descriptions and results cross from a server-controlled metadata plane into model planning and user approval.

8.5.3 Guided defensive review

1. Which metadata reaches the model or approval UI?
2. Does execution use the same reviewed, authenticated definition?

8.5.4 Defensive resolution

Authenticate servers, sanitise and canonicalise metadata, show security-relevant fields, and authorise every call outside the description.

Validation: Use benign instruction-like metadata to confirm it remains visible as data and cannot broaden a tool's permitted effect.

Gotcha: A harmless tool name can conceal instructions in descriptions, annotations, or returned content.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

8.6 `namespace-typosquatting` — Namespace Typosquatting

Category: Supply chain · **Difficulty:** ungraded · **Declared variants:** 1 **Catalog key:** `appsecco-mcp/namespace-typosquatting` · **Defensive pattern:** `dependency-confusion`

8.6.1 Challenge description

Review the Namespace Typosquatting scenario as implemented by Appsecco, focusing on its Supply chain boundary.

8.6.2 Educational focus

Dependency identity must bind namespace, registry, version, and digest rather than relying on the package name alone.

Trust boundary: A package name crosses internal and public registries where resolution priority may select an unintended publisher.

8.6.3 Guided defensive review

1. Which registry owns the namespace?
2. Can a public package or lookalike name outrank the intended internal dependency?

8.6.4 Defensive resolution

Use scoped private namespaces, explicit registries, lockfiles and hashes, signed provenance, and deny undeclared sources.

Validation: Resolve benign test packages against isolated registries and assert public, unsigned, or hash-mismatched candidates fail closed.

Gotcha: Pinning a version does not help when the same name and version can resolve from the wrong registry.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

8.7 `outdated-packages` — Outdated Packages

Category: Supply chain · **Difficulty:** ungraded · **Declared variants:** 1 **Catalog key:** `appsecco-mcp/outdated-packages` · **Defensive pattern:** `dependency-hygiene`

8.7.1 Challenge description

Review the Outdated Packages scenario as implemented by Appsecco, focusing on its Supply chain boundary.

8.7.2 Educational focus

Dependency age is not the decision; supported versions, reachable vulnerable behavior, provenance, and update policy are.

Trust boundary: A known-vulnerable or abandoned dependency crosses from the package graph into a network-facing AI or MCP service.

8.7.3 Guided defensive review

1. Which vulnerable components are reachable in this deployment?
2. What owner and deadline govern upgrade or compensating controls?

8.7.4 Defensive resolution

Maintain an SBOM and lockfile, scan advisories, remove unused packages, update supported versions, and document time-bound exceptions.

Validation: Use a synthetic advisory or policy threshold to confirm affected transitive dependencies block promotion with an actionable owner.

Gotcha: A clean direct-dependency list can hide an exploitable vulnerable version deeper in the resolved graph.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

8.8 `secrets-pii` — Secrets and PII Exposure

Category: Secrets/Exposure · **Difficulty:** ungraded · **Declared variants:** 1 **Catalog key:** `appsecco-mcp/secrets-pii` · **Defensive pattern:** `secrets`

8.8.1 Challenge description

Review the Secrets and PII Exposure scenario as implemented by Appsecco, focusing on its Secrets/Exposure boundary.

8.8.2 Educational focus

Secrets should remain outside prompts and should be disclosed only through explicit, authorised interfaces.

Trust boundary: Sensitive configuration or identity data crosses into a response, log, completion, or model context.

8.8.3 Guided defensive review

1. Where is sensitive data introduced?
2. Which output, error, metadata, or completion path can reveal it?

8.8.4 Defensive resolution

Use a secrets manager, minimise context, redact errors and logs, authorise reads, and monitor disclosure paths.

Validation: Test redaction and access decisions with synthetic canaries and assert that unauthorised callers receive no value.

Gotcha: Masking the final response is insufficient when the same value remains available through debug or metadata paths.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e

8.9 `wikipedia-streamable` — Wikipedia (Streamable HTTP)

Category: Remote content injection · **Difficulty:** ungraded · **Declared variants:** 1 **Catalog key:** `appsecco-mcp/wikipedia-streamable` · **Defensive pattern:** `prompt`

8.9.1 Challenge description

Review the Wikipedia (Streamable HTTP) scenario as implemented by Appsecco, focusing on its Remote content injection boundary.

8.9.2 Educational focus

Content provenance and instruction authority must remain distinct throughout the model pipeline.

Trust boundary: Untrusted content crosses into model-visible instructions or persistent context.

8.9.3 Guided defensive review

1. Which fields can influence instructions?
2. Can retrieved, tool, or remembered content persist or trigger an action?

8.9.4 Defensive resolution

Use typed instruction/data channels, provenance labels, output checks, scoped memory, and least-privilege tools.

Validation: Use inert policy fixtures to confirm untrusted text remains data and cannot alter an authorised workflow.

Gotcha: Filtering only the current user message misses instructions arriving through retrieval, tools, or memory.

Evidence status: Pinned upstream implementation; RedSec adapter health evidence only. Offensive solution: **not verified—outside the current scope**.

Provenance: https://github.com/appsecco/vulnerable-mcp-servers-lab/tree/47b7f2082261136b667daa7be9273698e66e479e